

Sommario

Titolo del progetto:.....	1
Gruppo.....	1
Abstract:.....	2
Introduzione (Problema)	2
Generazione del dataset iniziale (Dataset).....	3
Dataset sintetico	3
Workflow per la creazione del dataset.....	3
Creazione scena in Blender.....	4
Tool (estensione blender)	4
Script python.....	5
Considerazioni	6
Distribuzione dati.....	6
Creazione del modello (Metodi e Codice).....	6
Il problema principale: Domain Adaptation.....	9
Dataset per fine-tuning	9
Fine-tuning.....	10
Valutazioni.....	10
Performance su test	11
Esperimenti.....	12
Punti critici e possibili migliorie future	13
Distribuzione modello (Demo).....	15
Conclusioni.....	16

Titolo del progetto:

RELAZIONE PROGETTO MACHINE LEARNING: REGRESSIONE NUMERO TAZZE

Gruppo

Anno: 2026

Gruppo: 16

Membri: Alfio Palermo

Alfio Palermo

Matricola: 1000000618

Abstract:

Il presente lavoro affronta il problema dell'image counting, ovvero la stima del numero di oggetti presenti in una scena a partire da una singola immagine, concentrandosi sul conteggio automatico di tazze posizionate su una superficie piana. Data la difficoltà e il costo associati alla raccolta e annotazione di dati reali, il progetto esplora l'utilizzo di dati sintetici generati tramite Blender come principale fonte di addestramento. A tal fine è stato sviluppato un tool personalizzato in Python, integrato in Blender, in grado di generare automaticamente immagini RGB e maschere segmentate, dalle quali viene ricavata la ground truth per il task di counting.

Il problema viene affrontato mediante un approccio di regressione diretta basato su una rete neurale convoluzionale (CNN), che riceve in input l'immagine completa e restituisce una stima del numero totale di tazze presenti nella scena. Il modello è stato inizialmente addestrato su un ampio dataset sintetico e successivamente sottoposto a una fase di fine-tuning su un dataset di immagini reali, al fine di affrontare le problematiche di domain adaptation.

I risultati mostrano che l'approccio proposto è in grado di apprendere efficacemente dai dati sintetici, mentre evidenziano le difficoltà di generalizzazione su dati reali, sottolineando il ruolo cruciale della variabilità del dataset e aprendo a possibili sviluppi futuri.

Introduzione (Problema)

Il problema dell'image counting consiste nella stima del numero di oggetti presenti in una singola immagine, e rappresenta una sfida rilevante soprattutto in presenza di variabilità visiva, occlusioni e condizioni di illuminazione non controllate.

Uno dei principali limiti nello sviluppo di modelli affidabili per questo tipo di task è la disponibilità di dataset adeguati. La raccolta e l'annotazione manuale di immagini reali risulta spesso costosa, soggetta a errori e difficilmente scalabile. Per questo motivo, negli ultimi anni si è diffuso l'uso di dataset sintetici, generati artificialmente tramite motori grafici, che permettono un controllo completo delle scene e la produzione automatica di annotazioni perfette.

Il presente progetto si inserisce in questo contesto e ha come obiettivo principale la realizzazione di un sistema di image counting per il conteggio automatico di tazze su superfici piane, basato prevalentemente su un dataset sintetico generato tramite Blender. Il sistema di generazione del dataset è un'evoluzione di un tool sviluppato in precedenza per problemi di segmentazione semantica, che viene qui esteso, rinforzato e adattato per supportare il task di regressione visiva.

L'obiettivo principale del presente lavoro, tuttavia, non è il raggiungimento delle massime prestazioni possibili nel task di image counting. In letteratura esistono infatti approcci più complessi e generalmente più performanti, basati su tecniche di object detection, instance segmentation o stima tramite density map.

Il focus del progetto è invece posto sull'analisi dell'efficacia dei dati sintetici e sulle problematiche legate alla domain adaptation, valutando in che misura un modello addestrato prevalentemente su dati artificiali riesca a generalizzare su dati reali.

A tal fine, il problema viene affrontato mediante un approccio di regressione diretta, basato su una rete neurale convoluzionale (CNN) che riceve in input l'immagine completa e restituisce come output una stima del numero totale di tazze presenti nella scena. Sebbene questo approccio risulti meno preciso in scenari complessi, esso consente di isolare e analizzare il comportamento del modello in funzione delle caratteristiche della scena e della qualità del dataset sintetico utilizzato.

Attraverso una serie di esperimenti, verranno analizzate le prestazioni del modello, evidenziando sia i punti di forza dell'approccio proposto sia i suoi limiti.

Generazione del dataset iniziale (Dataset)

Dataset sintetico

Il dataset utilizzato nel progetto è stato generato artificialmente mediante Blender, sfruttando un tool di estensione sviluppato in Python e già utilizzato in precedenza per problemi di segmentazione semantica. Tale strumento è stato esteso e adattato al fine di supportare il task di image counting, consentendo la creazione automatica di scene tridimensionali contenenti un numero variabile di oggetti e la generazione simultanea delle relative annotazioni.

L'utilizzo di un dataset sintetico permette di superare alcune delle principali criticità associate ai dati reali, garantendo potenzialmente:

- controllo sul numero di oggetti presenti nella scena;
- variabilità controllata delle configurazioni spaziali;
- annotazioni precise e prive di errore umano;
- scalabilità nella produzione dei dati.

Ogni scena viene generata posizionando un numero variabile di tazze su una superficie piana, con configurazioni casuali ma controllate. In particolare, durante la generazione vengono introdotte variazioni riguardanti:

- il numero di tazze presenti nella scena, con un range compreso tra 0 – 18.
- la posizione e la rotazione degli oggetti;
- il tipo e il colore delle tazze;
- le condizioni di illuminazione;
- l'angolazione e la distanza della camera.
- Caratteristiche specifiche delle immagini e della camera.

Workflow per la creazione del dataset

I passi che servono per arrivare alla creazione del dataset sono:

1-Creazione scena in Blender

2-Utilizzo del plugin sviluppato

3-Script Python per contare gli elementi nell'immagine e creare file label.

Di seguito spiegati più approfonditamente.

Creazione scena in Blender

Sono state ricreate diverse scene ambientate in una cucina-salotto, all'interno delle quali è stato - posizionato centralmente un tavolo con sopra alcune tazze. Gli oggetti presenti nella scena sono stati generati a partire da immagini reali, dalle quali sono stati creati i relativi modelli 3D utilizzando tool online basati su intelligenza artificiale (in particolare Meshy.AI). Successivamente, i modelli sono stati rimodellati in Blender al fine di ridurre il numero di poligoni e renderli più adatti alla generazione massiva di immagini.

Tool (estensione blender)

Dentro Blender il plugin creato (in Python) permette di settare i parametri come da immagine



Figura 1- menù del plugin realizzato

Come si evince dall'immagine è possibile creare un “parallelepipedo virtuale” settando le coordinate spaziali MinX-MaxX, MinY-MaxY, MinZ-MaxZ, questo rappresenterà lo spazio 3D entro cui la camera in scena verrà posizionata in maniera randomica con angolazione randomica (entro dei range fissi).

Col parametro #Photos è possibile settare quante foto scattare in automatico.

Il parametro Random FOV introduce variazioni casuali nel campo visivo della camera, mentre Random DOF modifica la profondità di campo, influenzando la messa a fuoco e l'effetto di sfocatura.

Infine, è possibile settare il formato di immagine in output (png|jpg) e la risoluzione delle immagini.

Quando viene premuto il pulsante WRITER il tool inizia il suo funzionamento e scatta in maniera randomica le immagini.

Vengono quindi create le immagini non etichettate nella cartella tmp.

A questo punto premendo il pulsante READER vengono riscattate le stesse immagini (i parametri con cui vengono scattate le singole foto allo step precedente infatti vengono memorizzati in un file con estensione csv in modo da ri-simulare la stessa foto), ma con gli oggetti segmentati, colorando in automatico ogni oggetto di un colore diverso.

È stata prevista all'interno del plugin la possibilità di selezionare quali oggetti vanno colorati in nero (e quindi considerati come oggetti NON target, "sfondo") inserendo gli oggetti nella collection IGNORE_SEGMENTATION.

Script python

A partire da tali maschere viene quindi calcolato automaticamente il valore di riferimento (*ground truth*) per il task di counting, contando con uno script python (al di fuori di Blender) quanti colori diversi da nero (non etichettate) ci sono in ogni singola immagine.

ES:



Figura 2 - immagine generata



Figura 3- Immagine generata segmentata

Come si può notare nelle immagini sopra, le tazze sono colorate tutti con colori differenti, ed è quindi semplice con uno script Python contare quanti colori unici ci sono in ogni immagine.

Considerazioni

L'esecuzione del plugin per la generazione delle immagini è un task poco costoso in termini di tempo, i parametri che maggiormente influiscono sono quelli legati alla risoluzione dell'immagine e al numero di immagini, al crescere di questi, infatti, il tool impiegherà più tempo a scattare le foto.

Il maggior dispendio di tempo si presenta nella fase di realizzazione della scena e degli elementi che la popolano, ma con i tool IA che permettono di creare oggetti semirealistici partendo da una foto o da un'immagine questa pipeline è oggi molto più rapida.

Sono quindi stati creati otto "batch" (un pezzo di dataset per ogni esecuzione del tool) su scene leggermente diverse, dove si è cercato di cambiare oggetti di sfondo, colori, illuminazione, tazze, elementi di arredo della stanza e il loro posizionamento.

Distribuzione dati

Unendo i dati generati alla fine, cercando anche di bilanciare il numero di tazze presenti nell'immagini generate, la distribuzione ottenuta è stata la seguente:

Numero tazze in scena	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Numero Immagini nel dataset	1082	235	187	236	235	346	164	418	158	171	225	193	195	206	253	234	214	281	667

Anche se si è provato a bilanciare il dataset, come si può facilmente notare la distribuzione non è uniforme. Questo vedremo in conclusione, analizzando i punti critici e migliorabili, potrebbe essere uno dei motivi che ha influito negativamente nelle performance del modello finale.

Codice (come avviare il progetto)

A causa dell'elevata dimensione del dataset si è preferito allegare un link al dataset, con i dati in cloud.

Per quanto riguarda i checkpoint per lo stesso motivo non sono stati caricati sul repository; tuttavia, è possibile replicarli avviando il training.

I codici di training del modello sono stati divisi su due file `notebook_regressione.ipynb` e `FineTuning.ipynb`. Vanno avviati in sequenza prima file `notebook_regressione.ipynb` che allenerà il modello per 20 epoch sui dati sintetici (terremo poi fino alla 15esima) e poi `FineTuning.ipynb` che continuerà ad allenare il modello ma sui dati reali.

Nota: Nella cartella `src` sono contenuti gli script utili per la generazione del dataset e alcuni script utili utilizzati per la rinomina massiva di file.

Creazione del modello (Metodi e Codice)

In questa sezione verranno spiegate parti salienti del codice e i metodi utilizzati.

Per la scelta del modello di previsione si è optato per una CNN.

I motivi che hanno portato a questa scelta sono essenzialmente due. Il primo è di natura tecnica: per lavorare con le immagini, l'utilizzo di una CNN rappresenta l'approccio più diretto ed efficace, mentre l'alternativa avrebbe previsto l'estrazione manuale delle caratteristiche tramite metodi di computer vision "classici" (come SIFT o Bag of Words).

Il secondo motivo, altrettanto rilevante, è di carattere didattico e personale: la volontà di sperimentare direttamente l'uso delle reti neurali e approfondirne il funzionamento attraverso l'implementazione in PyTorch.

Per la realizzazione degli esperimenti svolti sono stati utilizzati Notebook Python, che hanno permesso una più semplice organizzazione.

Sinteticamente e schematicamente i passi effettuati sono stati i seguenti:

1-Installazione e importazione delle librerie necessarie.

2-Analisi della distribuzione del dataset (come spiegato sopra)

2-Definizione di CupsDataset: una classe Dataset di PyTorch che serve a:

- leggere le immagini delle tazze dal disco
- associare a ogni immagine il numero di tazze presenti
- restituire coppie (immagine, label) pronte per il training della rete

3-Calcolo media e deviazione standard in modo da normalizzare successivamente nelle transforms i dati.

4-Definizione delle transforms da applicare ai dati; in particolare ne sono state definite 2, una da usare in fase di inferenza che si limita a ridimensionare le immagini (qual ora fosse necessario) e a normalizzare i dati;

la seconda transform invece è stata usata per la fase di training e differisce rispetto alla prima in quanto sono state aggiunte delle variazioni random sulle immagini, in particolare:

- Ruota casualmente di ± 5 gradi.
- Rovescia orizzontalmente con probabilità 30%.
- Cambia colore/luminosità/contrasto/saturazione leggermente.
- Aggiunge un piccolo rumore casuale per rendere i dati più vari.

Ciò è stato fatto per introdurre variabilità e provare a rendere più "realistici" i dati sintetici, che sono spesso troppo piatti da questo punto di vista.

5-Suddivisione del dataset in training set (80%) e validation set(20%), per ciascuno sono state applicate le relative transform.

6- Definizione del modello:

Il modello è una rete neurale convoluzionale personalizzata. La parte iniziale, (feature extractor), è costituita da tre blocchi di convoluzione con ReLU e max pooling: le convoluzioni estraggono le caratteristiche principali dell'immagine, mentre il pooling riduce progressivamente la dimensione.

Dopo questa fase, l'output viene "appiattito" automaticamente in un vettore (flattening), in modo da poterlo passare agli strati fully-connected. La parte finale, il classifier, prende il vettore appiattito, lo passa attraverso uno strato con 256 neuroni e ReLU, e infine produce un unico valore come output.

```
def __init__(self, input_size=(3, 500, 500)):
    super(CustomCNN, self).__init__()

    # -----
    # Feature extractor
    # -----
    self.features = nn.Sequential(
        nn.Conv2d(3, 32, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2), # /2 -> 500 -> 250
        nn.Conv2d(32, 64, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2), # /2 -> 250 -> 125
        nn.Conv2d(64, 128, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2), # /2 -> 125 -> 62
    )

    # -----
    # Calcolo automatico dimensione flatten
    # -----
    with torch.no_grad():
        dummy = torch.zeros(1, *input_size) # batch 1, canali x H x W
        dummy = self.features(dummy)
        self.flatten_dim = dummy.numel() # totale elementi dopo conv

    # -----
    # Classifier
    # -----
    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(self.flatten_dim, 256),
        nn.ReLU(),
        nn.Linear(256, 1) # output regressione
    )
```

7-Scelta della Loss:

In una prima fase è stata utilizzata la Mean Squared Error (MSE), trattandosi di una funzione costo comunemente adottata nei problemi di regressione. Tuttavia, durante i primi esperimenti, è emerso come questo approccio portasse a un addestramento poco stabile e a predizioni eccessivamente influenzate dagli errori più grandi.

Considerata la natura del problema, è risultata più appropriata la Mean Absolute Error (MAE). La MAE misura l'errore in termini di differenza assoluta tra valore predetto e valore reale, rendendo la funzione costo direttamente interpretabile nel contesto del counting (ad esempio, un errore pari a 1

corrisponde a una tazza contata in più o in meno). Inoltre, a differenza della MSE, la MAE non amplifica quadraticamente gli errori, permettendo al modello di concentrarsi sull'accuratezza media delle predizioni lungo l'intero range di valori.

Nelle considerazioni finali verranno comunque discusse possibili alternative alla MAE, potenzialmente più adatte al problema di counting.

8-Checkpoint

Poiché i dati erano numerosi si è deciso di salvare i checkpoint ogni tot interval, questo ha permesso di effettuare molti esperimenti diversi, sperimentare con il modello e i suoi parametri, effettuare prove variando i parametri, riprendere training (vedremo successivamente la fase di fine-tuning).

Ad ogni checkpoint sono stati salvati i dati relativi al modello (pesi), gli storici dei valori di loss di training e di validation, l'epoca a cui si è arrivati e parametri dell'ottimizzatore.

9- Sono stati loggati i valori di loss sia di training che di validation su un file csv, inoltre sono state anche plottate in un grafico in modo da apprezzare visivamente l'andamento delle curve in funzione delle epoche.

Il training è stato arrestato prima della fine (early stopping) quando ci si è resi conto che la loss della validation iniziava a non scendere più mentre quella di training continuava a scendere (rischio overfitting).

I best value ottenuti per la loss del validation si sono attestati intorno a $MAE=1.36$.

Ovvero la rete in valore assoluto sbagliava più o meno di 1.36 tazze, un risultato promettente considerando il task e il range da 0 a 18 tazze, un range ampio.

Il problema principale: Domain Adaptation

Come previsto in fase di progettazione la vera sfida non era far funzionare la rete sui dati generati (sintetici) ma farla funzionare sui dati reali.

Da diverse prove svolte su foto scattate con lo smartphone è stato da subito evidente che le prestazioni sui dati reali erano pessime.

Proprio per questo è stata introdotta una fase di fine-tuning del modello, in modo da adattare il modello ai dati reali.

La fase di fine-tuning è stata implementata per comodità e pulizia su un notebook a parte.

Il problema in questi casi è sempre lo stesso, ed è proprio il problema che si vuole risolvere con i dati sintetici, ovvero avere molti dati reali etichettati.

Dataset per fine-tuning

Sono state scattate con lo smartphone 25 foto per ogni "classe" (numero di tazze in una immagine), ottenendo così $19 \times 25 = 475$ immagini reali, etichettate visivamente e manualmente singolarmente.

La produzione del dataset di immagini reali ha richiesto un impegno significativo in termini di tempo. Oltre alla fase di etichettatura manuale, che ha comportato un'attenta analisi visiva delle immagini, anche la realizzazione stessa degli scatti ha richiesto un notevole investimento di tempo, necessario per l'allestimento e la gestione di un set fotografico coerente e controllato.

Dopo aver creato questo nuovo dataset di immagini reali è stata riapplicata tutta la procedura creando stavolta training, validation e test set.

Fine-tuning

Prima di riprendere il training del modello ripartendo da checkpoint precedentemente salvato sono stati bloccati alcuni parametri della rete, in particolare:

```
for param in model.features.parameters():
    param.requires_grad = False # blocco tutto il backbone

for param in model.features[-1].parameters(): # ultimo conv attivo
    param.requires_grad = True # sblocco solo l'ultimo layer convoluzionale

#cambio optimizer
fine_tune_optimizer = torch.optim.Adam([
    {'params': model.features[-1].parameters(), 'lr': 1e-5}, # ultimo layer conv
    {'params': model.classifier.parameters(), 'lr': 1e-4} # head
], weight_decay=1e-5)
criterion = nn.L1Loss()
```

Il codice sopra impone alla rete di non aggiornare i pesi dei primi strati convoluzionali (il “backbone”), quindi mantiene intatte le caratteristiche già imparate. Poi riattiva l'allenamento solo per l'ultimo strato convoluzionale, così la rete può adattarsi un po' ai dati reali senza riscrivere da zero tutte le rappresentazioni già utili.

Con la seconda parte di codice invece stiamo imponendo di aggiornare solo l'ultimo layer convoluzionale con un learning rate molto basso $1e-5$, e la parte finale della rete classifier con learning rate più alto $1e-4$.

Il `weight_decay` che è stato aggiunto durante i vari esperimenti aiuta a evitare overfitting, penalizzando pesi troppo grandi (regularization L2).

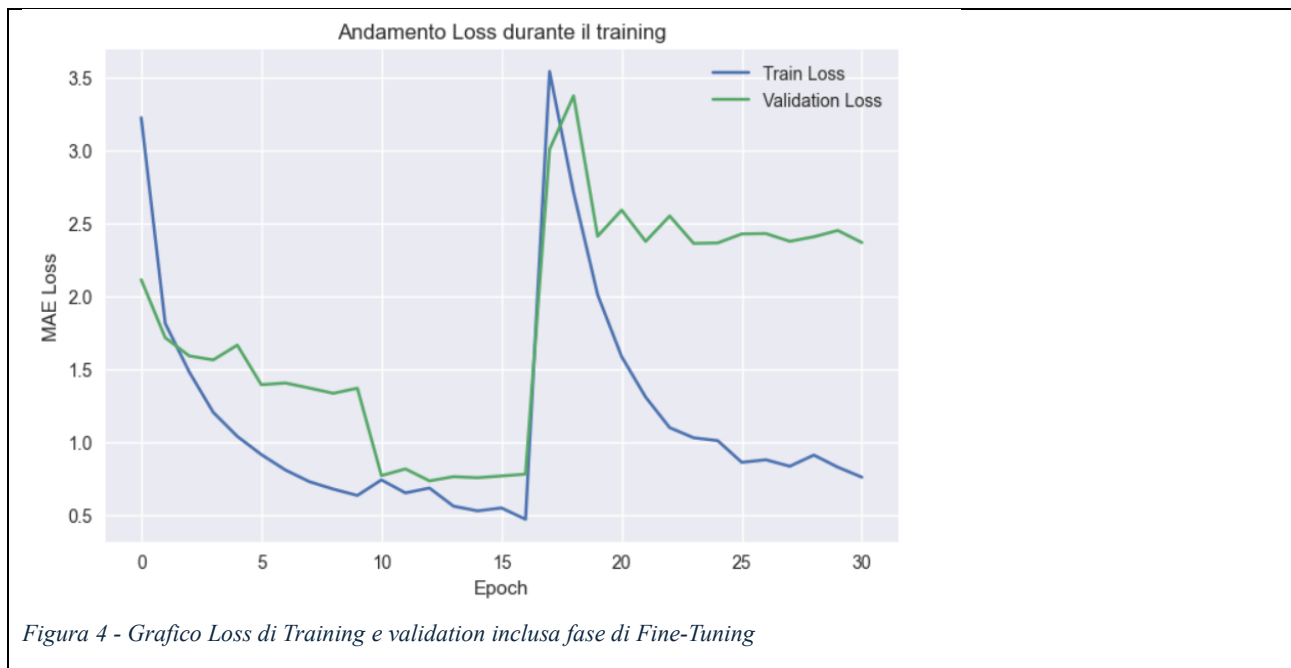
Per quanto riguarda la funzione costo è rimasta anche in fine-tuning MAE.

Valutazioni

Dopo alcune epoche si è visto che i valori migliori di validation si attestavano attorno a $MAE=2.35$.

Sono stati svolti diversi esperimenti ma non si è mai riusciti a scendere al di sotto di questi valori sul validation set.

Di seguito il plot delle loss di training e validation



Come si può notare dal grafico sopra, dopo la quindicesima epoca è presente un picco, questo è dovuto al fatto che il fine tuning è partito da epoca 16, cambiando quindi i dati di training, passando ai dati reali.

Performance su test

Le prestazioni del modello sul test set sono state valutate tramite diverse metriche di regressione. Il MAE (Mean Absolute Error) misura l'errore medio assoluto tra valori predetti e reali, fornendo un'indicazione immediata dell'errore medio commesso dal modello.

Il MSE (Mean Squared Error) penalizza maggiormente gli errori più grandi, risultando sensibile agli outlier, mentre il RMSE (Root Mean Squared Error) riporta tale misura nella stessa unità della variabile target, rendendone più intuitiva l'interpretazione.

Infine, il coefficiente di determinazione R^2 indica la capacità del modello di spiegare la varianza dei dati osservati, con valori più prossimi a 1 che indicano una migliore qualità predittiva.

I valori ottenuti sui dati reali di test (72 immagini) sono:

MAE : 2.5369

MSE : 10.3206

RMSE : 3.2126

R^2 : 0.7051

Per provare a migliorare le prestazioni è stato provato un metodo di Test-Time Augmentation-based averaging.

Per ogni immagine del test set vengono generate quattro versioni (immagine base, flip orizzontale, variazione di luminosità/contrasto e leggera rotazione).

Il modello produce una predizione per ciascuna versione e il risultato finale è ottenuto come media delle predizioni.

Questo metodo purtroppo non ha portato a prestazioni migliori rispetto alla regressione semplice su test set, ma anzi ne ha peggiorato la resa. La spiegazione è che questo metodo è solitamente più performante per il task di classificazione, dove si forma un sistema di voting, mentre nella regressione un errore grosso anche su una sola variante compromette e sporca il risultato della regressione.

MAE : 2.7108

MSE : 11.4416

RMSE : 3.3825

R^2 : 0.6731

Esperimenti

Come già esposto durante questo elaborato sono stati svolti diversi esperimenti, in primis sulla generazione delle immagini.

Arrivare a generare l'immagine segmentata con colori uniformi in maniera corretta è stato un processo lungo e pieno di prove. Ad esempio, si sono provati i diversi motori di rendering disponibili su Blender, si è poi scelto di escludere il motore cycles, che avrebbe generato immagini con luci più realistiche perché i tempi di realizzazione delle immagini diventavano troppo lunghi.

Un problema sfidante è stato capire come generare output omogenei nelle immagini di GroundTruth.

I colori infatti risultavano inizialmente sfumati, questo impediva il corretto conteggio dei colori unici nell'immagine e quindi conseguentemente il numero di tazze contenute.

Effettuando diverse prove si è poi trovato il parametro in Blender che andava correttamente settato in modo da non avere variazioni di colore nell'immagine finale (parametro retinatura del rendering da settare a 0).

Altri esperimenti sono stati fatti come già detto provando ad allenare il modello con diverse funzioni costo, MSE e MAE.

Altri esperimenti sono stati condotti sulla transform delle immagini di training.

Inizialmente le transform create erano funzionali all'adattamento dell'immagine alla rete neurale (un semplice resize essenzialmente).

Successivamente come precedentemente spiegato sono state introdotte delle transform per introdurre variabilità nelle immagini di training.

Durante il fine-tuning si è anche provato a variare e sbloccare più starti della rete, ma senza avere alcun particolare beneficio.

Punti critici e possibili migliorie future

Rianalizzando il progetto a conclusione diverse sono le migliorie attuabili e gli esperimenti ancora conducibili.

Sono possibili ulteriori margini di miglioramento nella fase di realizzazione del dataset. In particolare, il dataset avrebbe potuto beneficiare sia di un numero maggiore di immagini sia, soprattutto, di una maggiore variabilità delle scene generate.

Sebbene il tool sviluppato risulti essere efficiente, ha ancora dei grossi limiti, ad esempio la variabilità automatica è introdotta solo nei parametri relativi alla camera, non vi è nessuna variabilità automatica nella scena e negli elementi che contiene.

Il numero e le diverse tipologie di tazze utilizzate, i colori delle pareti, gli oggetti in più o in meno, la luce nella scena, sono tutti parametri che al momento vengono variati manualmente creando più scene, con i limiti che questo comporta.

Sarebbe utile per versioni future implementare sistemi che introducono variabilità automatica anche nella scena (ad esempio spostare tazze automaticamente, ruotarle, cambiarne il numero, le texture, etc.).

Ciò non è ancora stato realizzato per diversi motivi, in primis perché realizzare questo aspetto di gestione automatica della scena richiede parecchio studio e lavoro, secondo perché l'obiettivo iniziale era quello di realizzare un tool generico, che potesse poi funzionare anche per task differenti e per soggetti differenti, senza essere troppo specifico al task conteggio o al soggetto tazze.

Il tool realizzato infatti è valido anche per problemi di segmentazione semantica, e sono già stati creati anche script che realizzano bounding box degli oggetti, anche poligonali (quindi anche detection e instance segmentation), creando anche relativa etichettatura Yolo e COCO style. Creare specificatamente qualcosa per il task del conteggio tazze in qualche modo avrebbe sicuramente potuto migliorare il dataset e consecutivamente le prestazioni; considerato però il tempo impiegato per la creazione dei diversi script Python, l'integrazione con Blender, lo studio della pipeline, i test sui motori di rendering, si è ritenuto il dataset un lavoro sufficiente per il progetto di Machine Learning e un buon punto di partenza per eventuali sviluppi e progetti futuri.

Altra criticità nel dataset, è che non risulta facile controllare il numero di tazze generate.

Il problema risiedeva nel fatto che, anche qualora in una scena fossero posizionate, ad esempio, cinque tazze sul tavolo, non era garantito che tutte le immagini generate in quel batch contenessero effettivamente cinque tazze. In molti casi, a causa dell'inquadratura della camera, le tazze risultavano parzialmente o totalmente escluse dalla scena, producendo immagini con un numero inferiore di oggetti visibili

Le possibili soluzioni a questo problema sarebbero: creare una scena per ogni "classe", cioè 19 scene e inquadrare l'intero tavolo, il risultato avrebbe avuto però minore variabilità (ad esempio non si sarebbero ottenuti primi piani delle tazze); l'altra soluzione, abbastanza banale, che però è stata pensata a posteriori è quella di aumentare di molto il numero di immagini generate, prendere la "classe" con il numero di immagini minore e scartare le immagini in avanzo nelle altre classi.

Si perderebbero tanti dati generati, ma siccome il costo di aumentare le immagini generate non è esoso non sarebbe un problema.

Per quanto riguarda il modello, una possibile miglioria riguarda la funzione di costo: invece di utilizzare esclusivamente una MAE o una MSE, si potrebbe adottare una Huber Loss.

La Huber Loss rappresenta un buon compromesso tra MSE e MAE, in quanto per errori di piccola entità si comporta in modo quadratico, analogamente alla MSE, garantendo una maggiore stabilità del gradiente durante l'addestramento. Per errori più elevati, invece, la crescita della funzione di costo diventa lineare, come nella MAE, riducendo l'influenza degli outlier e prevenendo penalizzazioni eccessive dovute a pochi campioni difficili.

Grazie a questo comportamento ibrido, la Huber Loss risulterebbe particolarmente adatta a problemi di regressione su domini discreti e limitati, come nel nostro caso, in cui è importante mantenere un buon equilibrio tra precisione sugli errori piccoli e robustezza rispetto a quelli più grandi.

La Huber Loss è definita come:

$$L_{\delta}(e) = \begin{cases} \frac{1}{2}e^2 & \text{se } |e| \leq \delta \\ \delta \left(|e| - \frac{1}{2}\delta \right) & \text{se } |e| > \delta \end{cases}$$

Dove con e indichiamo l'errore con δ indichiamo un valore soglia. (ad esempio nel nostro caso potremmo impostare $\delta = 1.5$ (Cioè se sbaglia a predire più di una tazza e mezzo allora usiamo $\delta \left(|e| - \frac{1}{2}\delta \right)$ simile alla MAE)

Il concetto è riassumibile in:

errori piccoli → “voglio precisione” → ~MSE

errori grandi → “non esageriamo” → ~MAE

Loss	Errori piccoli	Errori grandi	Stabilità	Interpretabilità
MSE	Ottima	pessima (esplode)	X	X
MAE	Ok	ottima	V	V
Huber	Ottima	ottima	V	V

Altra miglioria per le prestazioni, implementabile come sviluppo futuro, potrebbe essere quella di creare un sistema di ensemble.

L'idea potrebbe essere quella di prendere i migliori N checkpoint (in termini di loss nella validation) e mediare i risultati di regressione (eventualmente pesando la bontà di ogni regressore).

Alfio Palermo

Matricola: 1000000618

Un'ulteriore possibilità, considerata, ma non esplorata per limiti di tempo, consiste nel riformulare il problema come un task di classificazione anziché di regressione. I dati, infatti, si prestano naturalmente a questo approccio, essendo il numero di tazze discreto e limitato a un insieme finito di classi.

Tra le sfide più rilevanti emerse durante lo sviluppo del progetto, una delle più interessanti ha riguardato il comportamento anomalo dei valori di loss. Al momento del passaggio dalla Mean Squared Error (MSE) alla Mean Absolute Error (MAE), la pipeline di training risultava già consolidata e il modello era stato addestrato correttamente per diverse epoche utilizzando la MSE. Non ci si aspettava quindi errori passando alla MAE; invece, la Loss sia del training che di validation alla prima epoca davano valori NaN come risultato.

La comprensione e risoluzione di questo problema ha richiesto non poco tempo al progetto, ma dopo varie ipotesi, debug (che è stato lasciato commentato nel notebook) il problema che è emerso è che la MAE non tollerava valori non normalizzati, a differenza di MSE. Infatti, modificando le transform in modo da normalizzare i dati, usando z-standardization (che inizialmente non era stata applicata perché con CNN non è sempre necessario) il problema è stato risolto.

La spiegazione scientifica che è stata trovata, è la seguente: MAE, essendo lineare, non riduce automaticamente l'impatto di grandi valori come fa la MSE. Questo ha portato a gradienti numericamente troppo grandi quando i dati non venivano normalizzati, causando valori NaN.

Distribuzione modello (Demo)

Una soluzione semplice per creare un'applicazione web e utilizzare il modello addestrato è quella di impiegare Gradio.

Gradio è una libreria Python che permette di costruire interfacce interattive in modo estremamente rapido, senza necessità di conoscenze approfondite di frontend. Tramite Gradio è possibile creare widget per input e output (come slider, caselle di testo, immagini, file audio), collegarli al modello e ottenere un'interfaccia web completamente funzionante con poche righe di codice.

Questo lo rende particolarmente adatto a progetti di prototipazione rapida o a contesti didattici, dove è utile mostrare risultati in tempo reale senza dover sviluppare necessariamente un'applicazione web complessa. Inoltre, Gradio si integra facilmente con altre librerie di machine learning come PyTorch, TensorFlow o scikit-learn, consentendo di mantenere un flusso di lavoro uniforme dall'addestramento alla distribuzione del modello.

Per il funzionamento basta avviare l'applicazione Demo.py su python (run), collegarsi in locale all'URL: <http://127.0.0.1:7860>.

L'interfaccia è minimale, permette di aggiungere un'immagine e premendo il tasto submit viene richiamata la funzione che effettua l'inferenza.

Nel repository è presente un video che mostra il funzionamento.



Conclusioni

Il modello sviluppato ha mostrato prestazioni soddisfacenti ma migliorabili: una previsione con un errore medio di circa 2–3 tazze risulta ragionevole, considerando la complessità del task e il range molto ampio (0-18). Un aspetto particolarmente interessante è stato ancora una volta dimostrare il potenziale di partire da dati sintetici per poi adattarsi a dati reali, aprendo la strada a future implementazioni più sofisticate che possano introdurre maggiore variabilità nel dataset.

Dal punto di vista didattico, il progetto ha permesso di approfondire numerosi strumenti e concetti che altrimenti sarebbero rimasti teorici: Blender e realizzazione di dati sintetici attraverso scripting Python e la comprensione pratica delle reti neurali.